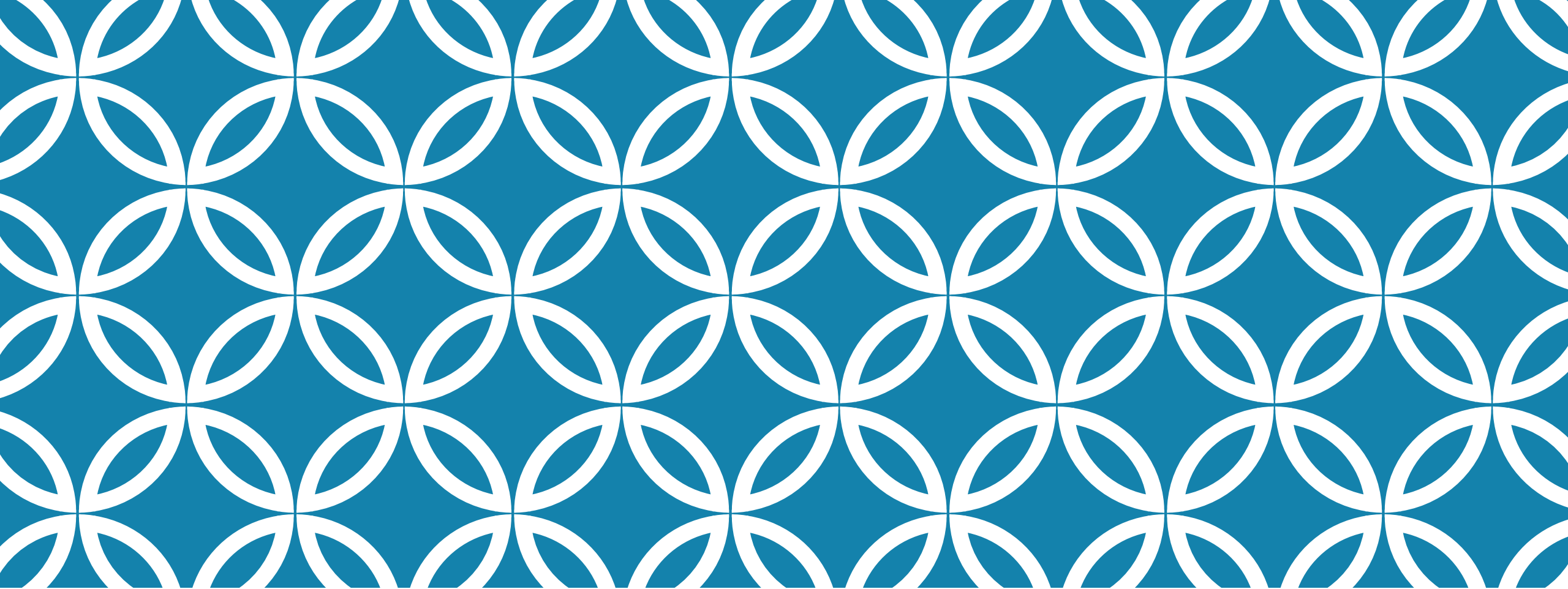




JAVASCRIPT 基础

从语法到数据处理工具

目标：独立写出一个跑在 Node.js 或浏览器控制台的数据处理工具

目录

一、基础语法

运行环境 · 语句与标识符 · var/let/const · 数据类型 · 类型转换 · 运算符 · 分支与循环

二、函数

函数声明与表达式 · 参数与返回值 · 作用域链与闭包 · 高阶函数

三、数据结构操作（核心模块）

数组方法全家桶 · 对象增删改查 · 对象遍历 · JSON

四、ES6+ 核心特性

模板字符串 · 箭头函数 · 解构赋值 · 扩展运算符 · 模块化

五、异步编程

Promise · then/catch 链 · async/await · 宏任务与微任务

六、调试与错误处理

try...catch · throw · 异步错误 · console · debugger · Call Stack

七、阶段实战任务

5 项要求 · 交付与自查 · 5 个推荐选题

目标

■ 一个真正能跑起来的数据处理工具：

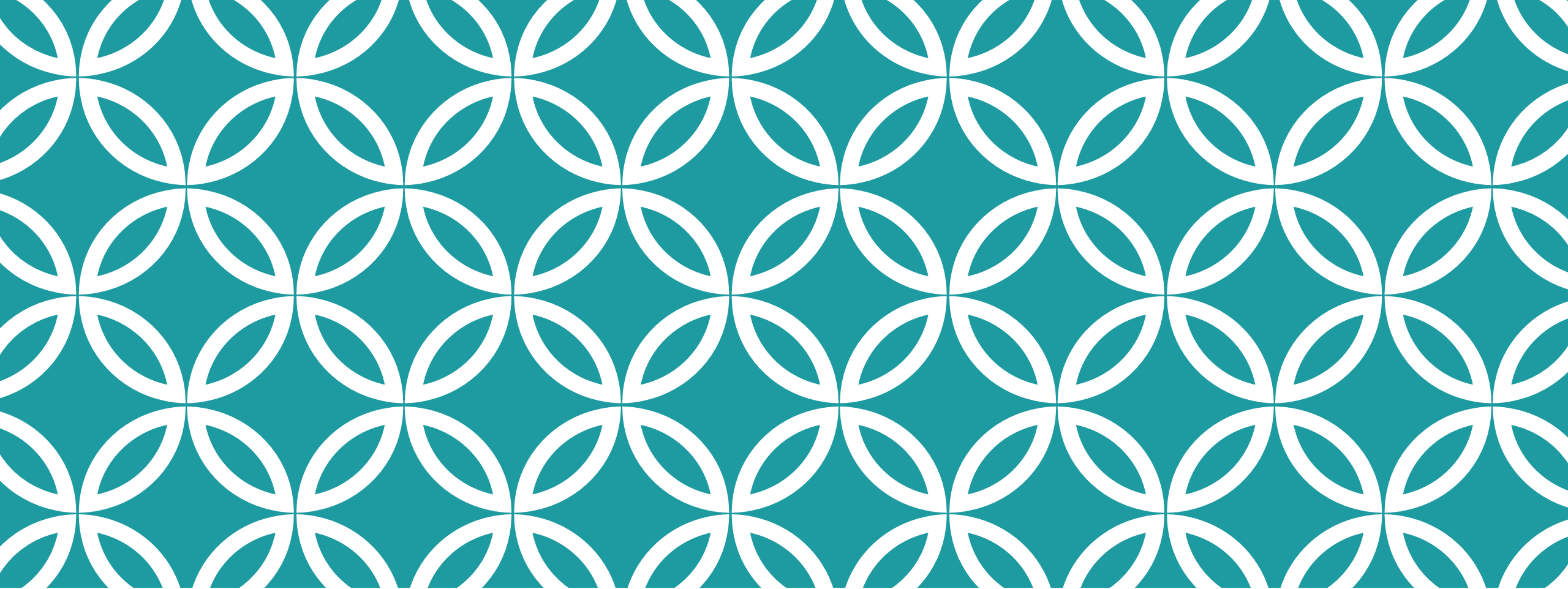
- 输入一组数据 → 筛选 / 映射 / 聚合 → 异步模拟加载 → 打印与排错

■ 三个"能"：

- 能跑：Node.js 或浏览器控制台，二选一，不依赖页面
- 能查：`console.table()` 把数据打印成表格，一眼看清字段
- 能排错：debugger 断点 + Call Stack + Scope 定位问题

■ 页面 DOM 操作、事件绑定、class 类 —— 留给后续模块





一、基础语法

目标：变量声明 + 类型判断 +
分支循环

JavaScript 是什么

■ Web 三大语言的分工：

- HTML 定义网页的内容 CSS 规定网页的布局 JavaScript 对网页的行为进行编程

■ JavaScript 与 Java 是完全不同的语言，不论概念还是设计

■ 1995 年由 Brendan Eich 发明；ES6（2015 年）是一次重大升级，本模块以 ES6+ 为主

■ 脚本语言：由 JS 引擎逐行解释执行，不需要编译成可执行文件

■ 本模块不涉及页面 DOM 操作，全部代码跑在控制台

一句话：HTML 决定有什么，CSS 决定长什么样，JS 决定做什么

运行环境（一）：Node.js

- 安装：nodejs.org 下载 LTS 版本，一路"下一步"即可
- 检查：终端执行 `node -v`，输出版本号（如 v20.11.0）即成功
- 运行：进入 .js 文件所在目录，执行 `node 文件名.js`
- HbuildX中运行.js文件

① 终端 (Terminal / PowerShell)

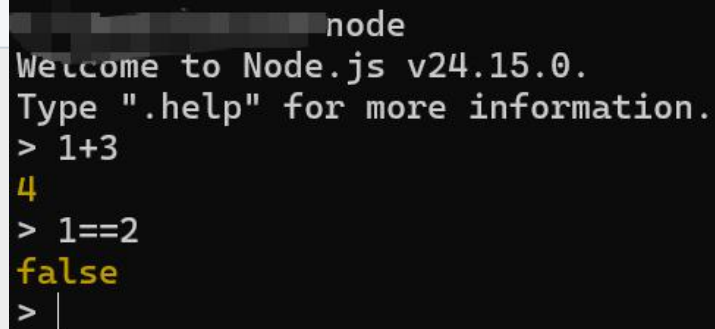
```
$ node -v
v20.11.0

$ cd D:\code\js-demo
$ node hello.js
Hello JavaScript
```

② 脚本文件 hello.js

```
// hello.js
console.log('Hello JavaScript');
```

// 直接在终端进入交互模式



```
node
Welcome to Node.js v24.15.0.
Type ".help" for more information.
> 1+3
4
> 1==2
false
> |
```

运行环境（二）：浏览器控制台

- 打开：Chrome / Edge 按 F12 → Console 面板，直接输入表达式回车即可执行
- 也可以写进 .js 文件，用 `<script src="app.js"></script>` 引入 HTML
- 脚本放在 `</body>` 之前：脚本编译会拖慢页面显示，放底部体验更好
- 外部 .js 文件内部不能再包含 `<script>` 标签；它会被浏览器缓存，加快加载
- 优点：零安装；缺点：刷新即清空，源码不在本地文件里

① index.html

```
<!DOCTYPE html>
<html>
<body>
  <h1>我的第一个 JS</h1>
  <!-- 脚本放在 body 结尾，避免阻塞渲染 -->
  <script src="app.js"></script>
</body>
</html>
```

#

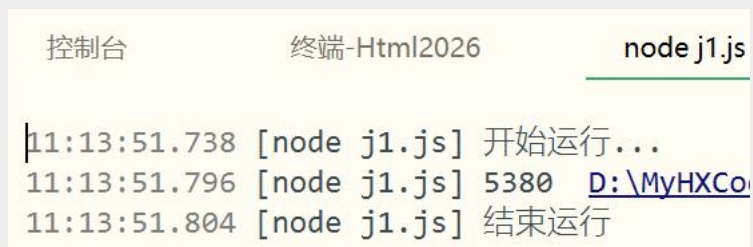
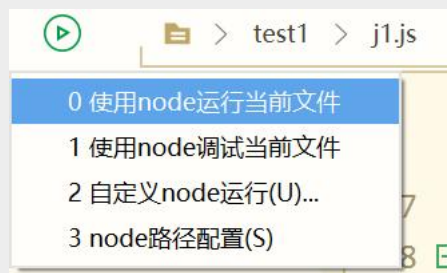
② app.js / Console 面板

```
// app.js
console.log('Hello JavaScript');

// Console 面板里可直接执行：
// > 1 + 2           → 3
// > '5' + 3         → '53'
```

运行环境（三）：HBuilderX

- Html项目中，新建JS文件
- 运行js文件或调试js文件（先点击js编辑页面，再点击三角运行键）
- 在控制台查看输出



语句、注释与标识符

- JS 程序由一系列语句构成，**分号分隔**；允许一行多句、一句多行
- 花括号 `{}` 组合成代码块，块内的语句一同执行
- 注释：单行 `//`，多行 `/* */` —— 注释不会被执行
- 标识符大小写敏感；推荐小驼峰 `userName`；不能用连字符（连字符留给减法）
- 建议：语句结尾统一加分号，缩进用 2 个空格

```
// 单行注释：这一行不会被执行
let price = 99;    let count = 2;    // 一行多句，不推荐这么写

/* 多行注释
   适合写文件头说明 */
let totalPrice = price * count;    // 小驼峰命名

if (count > 0) {    // 花括号 = 代码块
    console.log(totalPrice);    // → 198
}
```

变量声明：var / let / const

关键字	作用域	能否重复声明	能否重新赋值	变量提升
var	函数作用域	可以	可以	提升并初始化为 undefined
let	块级作用域 {}	不可以	可以	声明前访问 → TDZ 报错
const	块级作用域 {}	不可以	不可以	必须声明时赋值，其他同 let

```
for (var i = 0; i < 3; i++) {}
console.log(i);           // 3    var 没有块作用域，泄漏到外面

for (let j = 0; j < 3; j++) {}
console.log(j);           // ReferenceError: j is not defined

const user = { name: '小明' };
user.name = '小红';       // ☒ 允许：改属性，引用没变
user = {};                 // ☒ TypeError：给常量重新赋值
```

JS 执行一段代码（一个作用域）时分两步：

阶段一：创建/登记（编译阶段）：先扫一遍作用域里所有的声明，把它们登记到一个内部记录里。

阶段二：逐行执行（运行阶段）：再从上到下真正执行语句。

提升 (hoisting) 是阶段一发生的事——声明在阶段一就被登记了，所以阶段二里你在声明语句之前就能“看见”这个变量名。

口诀：默认用 const，需要重新赋值才用 let，永远不要再写 var

基本数据类型

- number: 整数与浮点数; Infinity、-Infinity、NaN (转换失败或无法运算)
- string: 单引号或双引号包裹, 索引从 0 开始; 数值 + 字符串 = 字符串
- boolean: true / false undefined: 声明过但未赋值 null: 空值, 需手动赋值
- JS 的动态类型: 同一个变量可以先后存放不同类型的值
- typeof 检测类型; typeof null → "object" 是历史遗留 bug, 记住即可

```
let a = 10, b = '10', c = true, d, e = null;
console.log(typeof a, typeof b, typeof c); // number string boolean
console.log(typeof d);                      // undefined
console.log(typeof null);                   // object (历史遗留)

console.log(Number('abc'));                // NaN
console.log(NaN === NaN);                  // false! 用 isNaN() 判断
console.log('Hello' + 7);                  // Hello7
```

类型转换与 == / ===

- 显式转换：Number() / String() / Boolean(), 主动调用，结果可控
- 隐式转换：JS 自动转，最容易踩坑 —— 加号遇字符串变拼接，减号一律转数字
- == 只比较值（会先隐式转换类型） === 值和类型都比较
- 假值只有 6 个：false / 0 / "" / null / undefined / NaN，其余都是真值

```
console.log('5' + 3);           // '53'    + 遇到字符串 → 拼接
console.log('5' - 3);           // 2        - 一律转数字
console.log(Number('12') + 1);  // 13       显式转换
console.log(Boolean(''));       // false    空字符串是假值

console.log('5' == 5);          // true     只比值
console.log('5' === 5);         // false    值和类型都比 ← 一律用这个
console.log(null == undefined); // true
console.log(null === undefined); // false
```

运算符与优先级

- 算术：+ - * / % (余数) ** (** 是幂运算, 2 ** 3 = 8)
- 比较：> < >= <= == === != !== 逻辑：&& || ! (与或运算属于短路求值；返回"决定结果"的那个值本身，而不是像C和Java一样返回 true/false)
- 三元：variablename = (condition) ? value1 : value2
- 优先级（高 → 低）：括号 () > 一元 > 算术 > 比较 > 逻辑 > 三元 > 赋值

```
let age = 17;
let tip = (age < 18) ? '未成年' : '已成年';    // '未成年'

console.log('' || '匿名');                    // '匿名' 空串是假值，取右侧
console.log('Tom' && 80);                      // 80    左侧为真，取右侧

console.log(2 + 3 * 4);                       // 14    先乘后加
console.log((2 + 3) * 4);                     // 20    括号最高
console.log(10 % 3, 2 ** 3);                  // 1 8
```

分支结构：if 与 switch(注意圆括号花括号)

- `if (条件) {}` 两分支 `if...else` 多分支 `if...else if...else` 注意圆括号花括号
- `switch (表达式) { case 值: 语句; break; ... default: 语句 }`
- ⚠ `case` 后漏写 `break` 会"穿透"执行下一个 `case` (有时是故意的)
- 右花括号 `}` 起到了分号的作用，不能省略

```
let score = 85;
if (score >= 90) { console.log('优秀'); }
else if (score >= 60) { console.log('及格'); }    // 及格
else { console.log('不及格'); }

switch (Math.floor(score / 10)) {
  case 10:
  case 9:  console.log('A'); break;    // 漏 break 会穿透到下面
  case 8:  console.log('B'); break;    // B
  default: console.log('C');
}
```

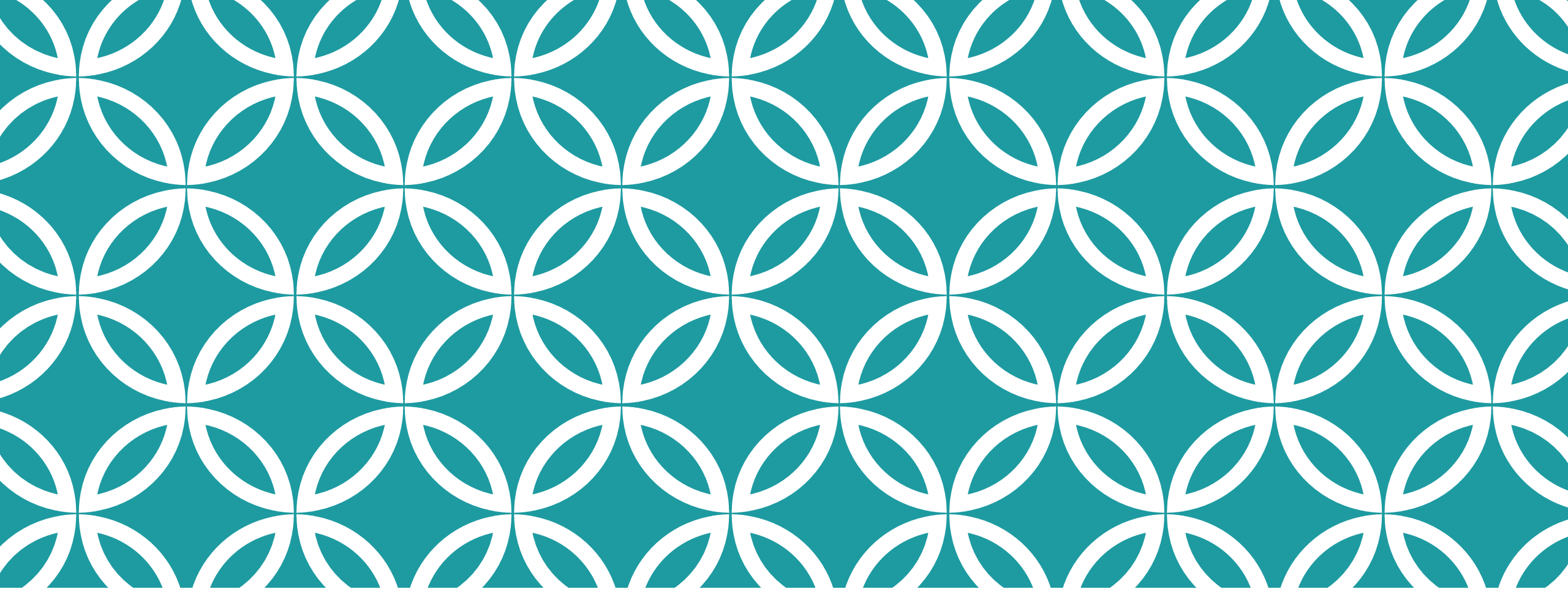
循环结构：for / while / do...while

- for (初始; 判断; 更新) {} —— 次数已知时用; while (条件) {} —— 先判断后执行
- do {} while (条件) —— 先执行一次再判断, 至少执行一次
- for...in 遍历对象的键 (模块三细讲); 数组推荐 for...of 直接取值
- break 结束整个循环, 执行循环体后的语句; continue 跳过本次, 进入下一次

```
for (let i = 0; i < 3; i++) {  
  if (i === 1) continue;    // 跳过本次  
  console.log(i);          // 0 2  
}
```

```
let n = 0;  
while (n < 3) { n++; }      // 先判断后执行, 结束时n=2  
do { n++; } while (n < 5);  // 至少执行一次, 结束时n=4
```

```
for (let i = 0; i < 10; i++) { if (i > 5) break; } // 提前结束, 跳出前i=6; 这里i仅在for语句中有效
```



二、函数

学习目标：带参数的复用逻辑，
并理解回调是怎么来的

函数声明与函数表达式

- 函数声明: `function 名字() {}` —— 会提升, 可以先调用后定义, 先调用后定义、递归、调试可读性
- 函数表达式: `const f = function () {}` —— 不提升, 必须先定义后调用 (这里的f指函数表达式的名称), 常用于赋值给变量、作为回调、作为参数, 配合箭头函数 `const f = () => {}` 更简洁
- 匿名函数 (没有名字) 常作为回调传给别的方法, 模块三会大量使用
- 函数只有被调用时才执行; 定义多少次都不消耗运行时间

```
// ① 函数声明 — 提升, 可以先调用后定义
console.log(add(2, 3));           // 5
function add(a, b) { return a + b; }

// ② 函数表达式 — 不提升, 必须先定义
const sub = function (a, b) { return a - b; };
console.log(sub(5, 2));           // 3

// ③ 匿名函数作回调
[1, 2, 3].forEach(function (n) { console.log(n); });
```

参数、默认参数与返回值

- 形参是定义时的占位符，实参是调用时真正传进去的值
- 默认参数：function f(a = 1) {}, 调用时不传就用默认值
- 基本类型（number / string / boolean）按值传递：函数内改不影响外部
- 引用类型（对象 / 数组）按引用传递：函数内改属性，外部跟着变
- return 返回结果；没写 return 的函数返回 undefined

```
function greet(name = '同学') {           // 默认参数
  return `你好, ${name}`;
}
console.log(greet());                     // 你好, 同学

let x = 10;
function changeNum(v) { v = 99; }
changeNum(x); console.log(x);             // 10   按值：原值不变

let user = { age: 18 };
function changeObj(o) { o.age = 20; }
changeObj(user); console.log(user.age); // 20   按引用：属性被改
```

作用域、作用域链与闭包

- 全局作用域：最外层，到处能访问；函数作用域：函数内声明，外面访问不到
- 块级作用域：let / const 只在 {} 内生效
- 作用域链：本层找不到变量，就一层层向外找，直到全局；都没有则 ReferenceError
- 闭包：函数记住并访问"定义时所在作用域"的变量，即使外层函数已经返回。

```
function f() {  
  let n = 0;  
  n++;  
  return n;  
}  
f(); // 1  
f(); // 1 (每次都重新创建 n, 从 0 开始)
```

```
let g = 'G';  
function outer() {  
  let o = 'O';  
  return function inner() { console.log(g, o); }; // 沿链向外找  
outer(); // G O; 注意outer()返回的是inner(), 而第二个()则是调用inner这个函数
```

```
function counter() { // 经典闭包：计数器  
  let n = 0; // 创建局部变量n  
  return () => ++n;  
  // 创建箭头函数，其函数体里引用了 n，返回出去的箭头函数仍然引用着 n。只要这个函数还活着（被 next 变量引用着），它依赖的 n 就不能被回收——否则函数执行时会找不到变量。  
}  
const next = counter();  
console.log(next(), next()); // 1 2 不是复制n的值，而是保留了对那个变量本身的引用  
//JS 引擎的垃圾回收规则：如果一个值还能被访问到（有引用链指向它），就不回收；完全没人引用了，才回收。
```

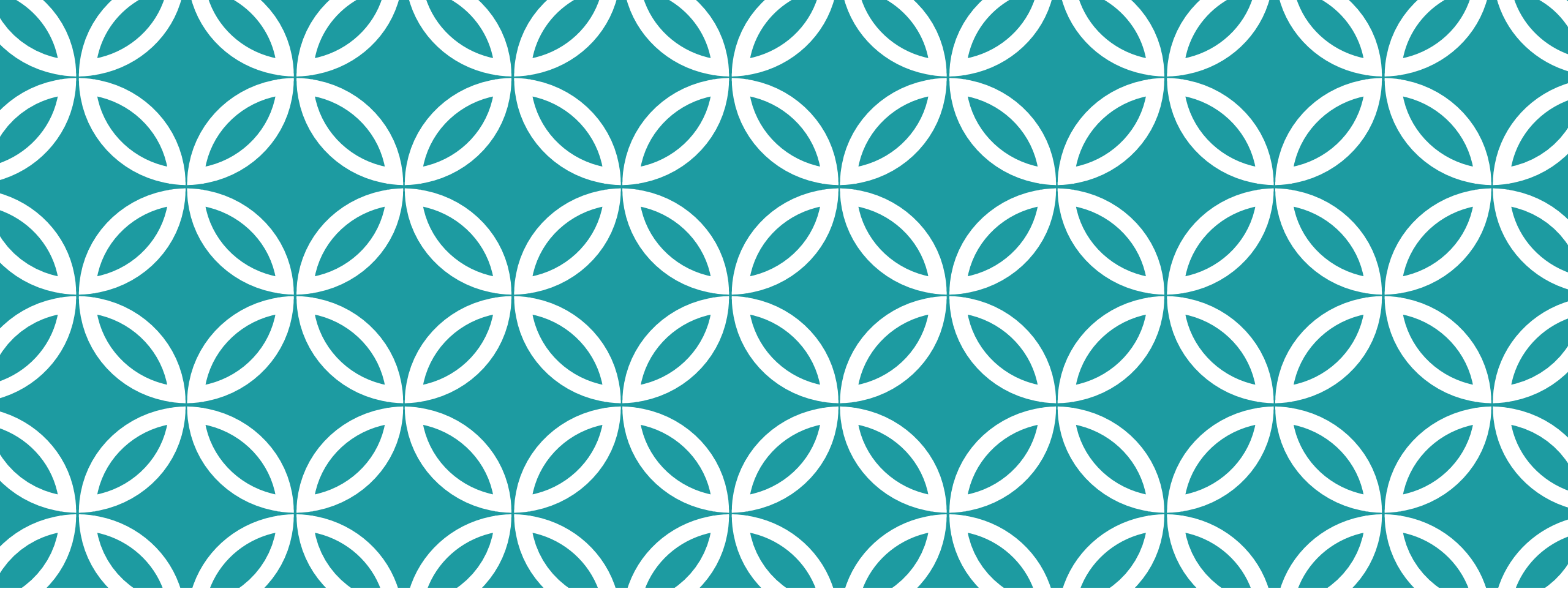
高阶函数入门

- 高阶函数 = 接收函数作为参数，或者返回一个函数的函数
- JS 中函数是"一等公民"：能存进变量、能当参数传、能当返回值
- 意义：把"做什么"和"怎么做"分开 —— map / filter / reduce 的回调就是高阶函数
- 这一页是模块三的前置：看懂回调，才看得懂数组方法

```
// ① 函数作为参数
const withTax = (fn) => (x) => fn(x) * 1.13;
const double = (x) => x * 2;
console.log(withTax(double)(100));      // 226

// ② 函数作为返回值（记住了 k）
const makeMul = (k) => (n) => n * k;
const times3 = makeMul(3);
console.log(times3(7));                  // 21

// ③ 最常见的形态：数组方法的回调
[1, 2, 3].map((n) => n * n);              // [1, 4, 9]
```



三、数据结构操作

核心模块：map / filter / find /
reduce —— 作业的主战场

数组基础与变异方法

- 数组用`方括号`定义，索引从 0 开始；逗号分割元素，JS 中`数组即对象`，`length` 是长度
- 会改变原数组（变异）：`push` / `pop` / `shift` / `unshift` / `splice` / `sort` / `reverse`
- 返回新数组（不变异）：`slice` / `concat` / `map` / `filter`
- ⚠ `sort()` 会改变原数组，且默认按字符串排序，数字排序要传比较函数

```
[10, 2, 1].sort(); // [1, 10, 2] ← 按字符串 "1" < "10" < "2"
```

```
const books = ['三体', '活着'];
books.push('小王子');           // 末尾加 → 3 (返回新长度)
books.pop();                    // 末尾删 → '小王子'
books.unshift('追风筝的人');    // 开头加 → 3
books.shift();                  // 开头删 → '三体'

const nums = [3, 1, 2];
nums.slice(0, 2);               // [3, 1] 不改原数组, 左闭右开
nums.sort();                    // [1, 2, 3] 改了原数组!
nums.sort((a, b) => b - a);      // [3, 2, 1] 数字降序要传比较函数
nums.sort((a, b) => a - b);      // [1, 2, 3] 数字升序要传比较函数
```

遍历与映射：forEach 与 map

- forEach：逐个遍历，只做副作用（打印、累加外部变量），返回 undefined
- map：一一映射，返回一个等长的新数组，不改变原数组
- 判断标准：需要拿到结果 → 用 map；只是想让每个元素"做件事" → 用 forEach

```
const goods = [  
  { name: '笔记本', price: 5000, stock: 3 },  
  { name: '鼠标',    price: 80,    stock: 0 },  
  { name: '键盘',    price: 300,   stock: 12 },  
];  
  
goods.forEach((g) => console.log(g.name));    // 只打印, 返回 undefined  
  
const names = goods.map((g) => g.name);        // ['笔记本','鼠标','键盘']  
const cards = goods.map((g) => `${g.name}: ¥${g.price}`);  
console.log(goods.length);                    // 3 原数组没变
```

筛选与查找：filter / find / some / every

- filter: 符合条件的元素组成的新数组 (一个都没有就是 [])
- find: 第一个符合条件的元素 (找不到是 undefined, 不是 null) ; findindex(), 返回该元素的索引值
- some: 有没有至少一个符合 → true / false every: 是否全部符合 → true / false

```
// goods 同上页: 笔记本(5000,3) / 鼠标(80,0) / 键盘(300,12)
goods.filter((g) => g.stock > 0);      // [笔记本, 键盘]    → 数组
goods.find((g) => g.price > 1000);     // {name:'笔记本',...} → 单个对象
goods.some((g) => g.stock === 0);     // true    有缺货
goods.every((g) => g.stock > 0);       // false   并非全有货

// 找不到时 (写判断条件务必区分)
goods.find((g) => g.price > 99999);    // undefined
goods.filter((g) => g.price > 99999);  // [] (空数组, 不是 null)
```

聚合计算：reduce

- `reduce()`：把数组里的所有元素，通过一个函数，从左到右"合并"成一个值。**reduce在这里不是"减少"，而是"归约"**
- 语法：`arr.reduce((累加器, 当前元素, 索引, 数组) => { ... }, 初始值)` *红色部分表示回调函数；第一个参数表示累加器acc：上一次回调返回的结果，会被"接力"传下去；第二个参数是当前元素；当前元素：本次正在处理的数组元素；*
- 累加器保存上一轮的返回值；初始值（累加器起点）一定要写，否则空数组会直接报错；

```
const nums = [5, 2, 9, 2];
nums.reduce((sum, n) => sum + n, 0);           // 18 求和；箭头函数单表达式自动 return
nums.reduce((sum, n) => sum + n, 10);           // 28 初始值为10的求和
nums.reduce((max, n) => (n > max ? n : max), nums[0]); // 9 最大值
nums.reduce((acc, cur) => {if (!acc.includes(cur)) acc.push(cur); return acc;}, []); // [5, 2, 9], 去重

// 按库存状态分组计数 → { 有货: 2, 缺货: 1 }
goods.reduce((acc, g) => {
  const key = g.stock > 0 ? '有货' : '缺货';
  acc[key] = (acc[key] || 0) + 1; // 没出现过就从 0 开始
  return acc; // 别忘了 return 累加器, reduce 靠回调的返回值来更新累加器的值
}, {});
```

对象：增删改查与嵌套访问

```
const person = {  
  name: 'Tom',      // 标签是 name, 里面放 'Tom'  
  age: 18,          // 标签是 age, 里面放 18  
  greet: function () { console.log('hi'); } // 标签是 greet, 里面放一个函数  
};
```

- 对象是"被命名值的容器"：用**花括号**定义。属性（property）是 name:value 对，存在属性里的函数叫方法，由逗号分隔。（当函数被放在对象里当作属性值时，我们习惯叫它方法method）。
- **属性名**一定是字符串,**属性值**可以是字符串、数字、布尔、数组、对象、函数
- 两种访问：obj.prop 与 obj["prop"]（键是变量或含特殊字符时用方括号）
- 增 obj.x = 1 删 delete obj.x 改 obj.x = 2 查 obj.x
- 访问方法必须带括号：obj.method(), 不带括号拿到的是函数定义本身
- 数组相当于在对象的基础上，加了一层"有序列表"的约定和一批专用方法,是"被特殊化的对象"。

```
const student = {  
  name: '小明', scores: { math: 92, english: 78 }, pass: true,  
};  
console.log(student.name, student['name']); // 小明 小明  
student.age = 18; delete student.pass; // 增 / 删  
console.log(student.scores.math); // 92
```

对象遍历：for...in 与 Object.keys()

- for...in 遍历对象的键（含原型链上的可枚举属性）
- Object.keys(obj) 只返回"自有可枚举"的键组成的数组 —— 推荐用它
- 配套：Object.values() 取值数组，Object.entries() 取 [键, 值] 数组
- 数组遍历推荐 for...of：直接拿到元素，而不是索引

```
const book = { title: '三体', price: 39, stock: 5 };

for (const key in book) { console.log(key, book[key]); }
// title 三体 / price 39 / stock 5

Object.keys(book);      // ['title', 'price', 'stock']
Object.values(book);    // ['三体', 39, 5]
Object.entries(book);   // [['title', '三体'], ['price', 39], ...]

for (const g of goods) { console.log(g.name); }    // 数组用 for...of
```

JSON：数据交换格式

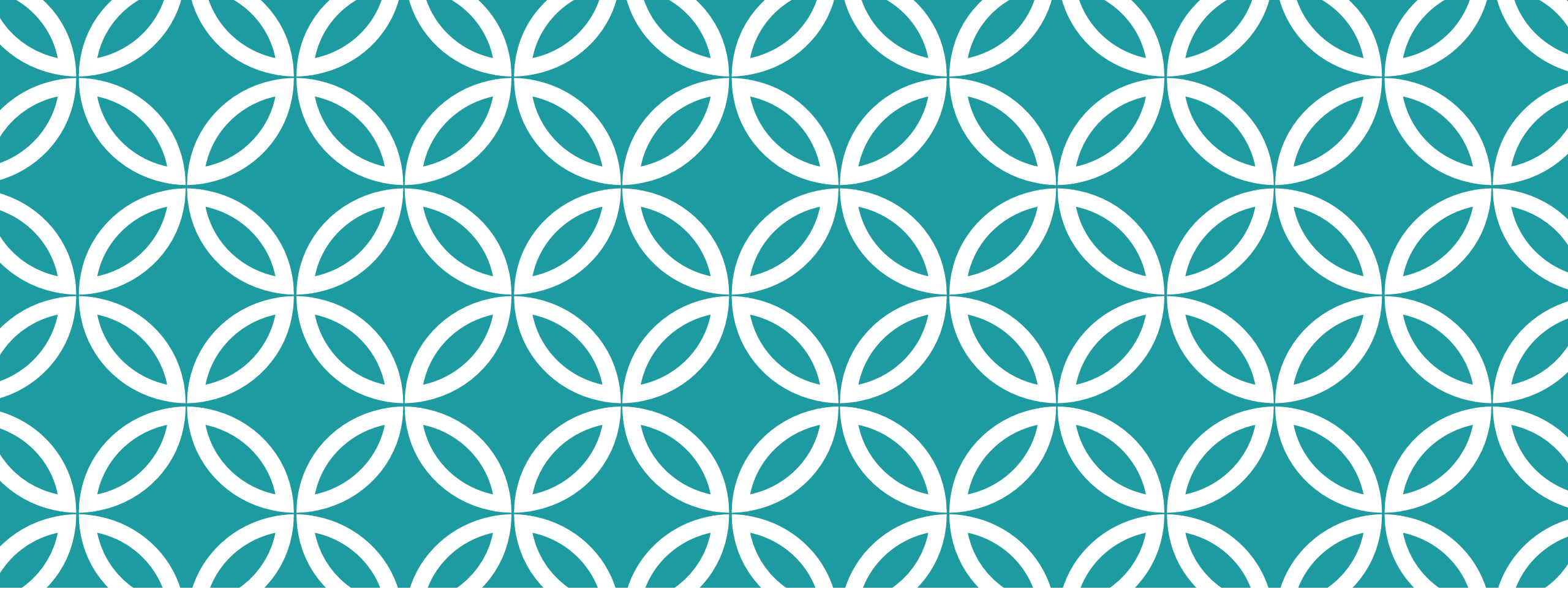
- JSON = JavaScript Object Notation，轻量级、独立于语言的数据交换格式
- 语法规则：名称/值对、逗号分隔、{} 存对象、[] 存数组、数组可包含对象
- 与 JS 对象的区别：键必须双引号、不能写注释、不能放函数
- JSON.stringify(对象) → 字符串（发请求用） JSON.parse(字符串) → 对象（收响应用）

```
const book = { title: '三体', price: 39, inStock: true };

const str = JSON.stringify(book);
console.log(str);           // {"title":"三体","price":39,"inStock":true}
console.log(typeof str);    // string

const obj = JSON.parse(str);
console.log(obj.title);     // 三体

JSON.parse('{ bad json }'); // SyntaxError → 必须用 try...catch 包住
```



四、ES6+ 核心特性

ES6 = 2015 年的重大版本

模板字符串

- 反引号 ` 包裹；`\${}` 里可以放任意表达式，不只是变量名
- 支持多行字符串：直接回车换行，不用再写 \n 和加号
- 对比加号拼接：可读性提升明显，引号套引号的错误也少了
- 作业要求：至少 3 处模板字符串

```
const name = '三体', price = 39, stock = 5;

// 加号拼接 (旧写法, 引号套引号容易错)
const s1 = '《' + name + '》售价 ' + price + ' 元, 库存 ' + stock + ' 本';

// 模板字符串 (推荐)
const s2 = `《${name}》售价 ${price} 元, 库存 ${stock} 本`;

// 多行 + 表达式 + 三元
const card = `
  书名: ${name}
  折后: ¥${(price * 0.8).toFixed(2)}
  状态: ${stock > 0 ? '有货' : '缺货'}
`;
```

箭头函数与词法 this (难点)

- 用 `=>` 定义。它本质上是“函数表达式”，不是函数声明。箭头函数最大的特点不是“写法短”，而是：没有自己的 `this`、`arguments`，不能作为构造函数，不能用作生成器。语法：参数 `=>` 返回值
- 核心差异：箭头函数没有自己的 `this`，它继承外层作用域的 `this`（词法绑定），在“出生”时就定下了。
- 普通函数的 `this` 在调用时才决定 —— 作为回调传出去，`this`就丢了
- 不适合用箭头函数的场景：对象方法需要 `this`、构造函数

```
const sayHi = () => {  
  console.log('Hi');  
}; //无参  
//单参，括号可省略  
const double = x => x * 2;  
//多参，必须括号  
const add = (a, b) => a + b;  
//单表达式：可省略 {} 和 return  
const square = x => x * x;  
//多行语句：需要 {} 和 return  
const calc = (a, b) => {  
  const sum = a + b;  
  return sum * 2;  
};
```

```
//返回对象字面量：要用括号包起来  
const createUser = name => ({ name });  
console.log(createUser2('Alice')); // { name: 'Alice' }  
//默认参数  
const greet = (name = '世界') => `你好, ${name}`;  
console.log(greet()); // 你好, 世界  
//剩余参数  
const sum = (...nums) => nums.reduce((a, b) => a + b, 0);  
console.log(sum(1, 2, 3)); // 6  
//解构参数  
const getName = ({ name }) => name;  
console.log(getName({ name: 'Tom', age: 18 })); // Tom
```

箭头函数的 `this` 沿作用域链向外找，普通函数的 `this` 看谁调用它

//下例中，箭头函数“出生”在 start 方法里面。
start 被 counter 调用，所以 start 的 this 是 counter。
箭头函数继承 start 的 this，所以它的 this 也是 counter。

```
const counter = {  
  count: 0,  
  
  start() {  
    setInterval(() => {  
      this.count++;  
      console.log(this.count); //从1打印到1000  
    }, 1000);  
  }  
};  
counter.start();
```

//例子中的箭头函数虽然写在 obj 里面，但它的“外层作用域”不是 obj，而是全局作用域。因为在 JavaScript 里，对象字面量的 {} 不会创建新的 this 环境。obj 只是一个对象，不是函数作用域，也不是 this 的绑定环境。

```
const obj = {  
  name: 'obj',  
  regular: function() {  
    console.log(this.name);  
  },  
  arrow: () => {  
    console.log(this.name); 这里 arrow 定义时外层是全局，所以 this 是全局，即undefined  
  }  
};  
obj.regular(); // obj  
obj.arrow(); // undefined (或外层 this.name)
```

this——当前上下文对象

- this是 JavaScript 关键字。this 是函数运行时，自动指向“当前调用者”或“当前上下文”的引用。
- this用途：让函数能访问当前操作的对象，实现复用
- 规则：谁调用函数，this 通常就是谁；new 时是新对象；call/apply/bind 可手动指定。

解构（Destructuring）赋值

- 解构 = 从数组或对象里“拆”出值，赋给变量。它不改变原数据，只是换一种更简洁的取值方式。形式上表示为`[]`或`{}`出现在赋值号`=`的左边。
- 数组解构：`const [a, b] = arr`，可以跳位、可以一行交换两个变量
- 对象解构：`const { title, price } = obj`，可重命名 `{ title: bookName }`、可给默认值
- 函数参数也能直接解构，调用时传整个对象即可

```
// 数组解构：跳过中间项、交换两数
const [first, , third] = ['a', 'b', 'c'];           // first='a' third='c'
let x = 1, y = 2;   [x, y] = [y, x];                 // x=2 y=1

// 对象解构：重命名 + 默认值
const book = { title: '三体', price: 39 };
const { title: bookName, stock = 0 } = book;        // bookName='三体' stock=0

// 函数参数解构
function show({ title, price }) { console.log(`${title}: ¥${price}`); }
show(book);                                           // 三体: ¥39
```

扩展运算符 ...

- 数组：[...a, ...b] 合并，[...a] 浅拷贝。更方便地展开、收集、组合参数/数组/对象，再配合 async/await、Promise.all 等完成异步控制。
- 对象：{ ...base, price: 99 } 合并，写在后面的覆盖前面的
- ⚠ 浅拷贝陷阱：只复制第一层，嵌套对象仍然共享同一份引用

```
const a = [1, 2], b = [3];
const merged = [...a, ...b];           // [1, 2, 3]

const base = { title: '三体', price: 39 };
const onSale = { ...base, price: 29 };  // 覆盖 price → 29

// 浅拷贝陷阱：嵌套对象仍共享引用
const o1 = { info: { stock: 1 } };
const o2 = { ...o1 };
o2.info.stock = 99;
console.log(o1.info.stock);             // 99 ⚠ 原对象也被改了

const sum = (...args) => args.reduce((s, n) => s + n, 0); // 剩余参数
```

模块化：export / import

- 一个 .js 文件就是一个模块，模块内的变量默认不外泄，避免全局污染
- 具名导出： `export const f = ...` 导入 `import { f } from './util.js'`
- 默认导出： `export default ...` 导入 `import f from './util.js'`（名字可自取，这个f不用定是原来的名字）
- 一个模块只能有一个 `export default`（所有可以任意取名，但可以有任意多个命名导出）
- Node.js 中使用 ES 模块：package.json 里加 `"type": "module"`，或改用 .mjs 扩展名

① 导出 utils.js

```
// ----- utils.js -----
export const TAX_RATE = 0.13;
export function withTax(p) {
  return p * (1 + TAX_RATE);
}
export default function fmt(n) {
  return `¥${n.toFixed(2)}`;
}
```

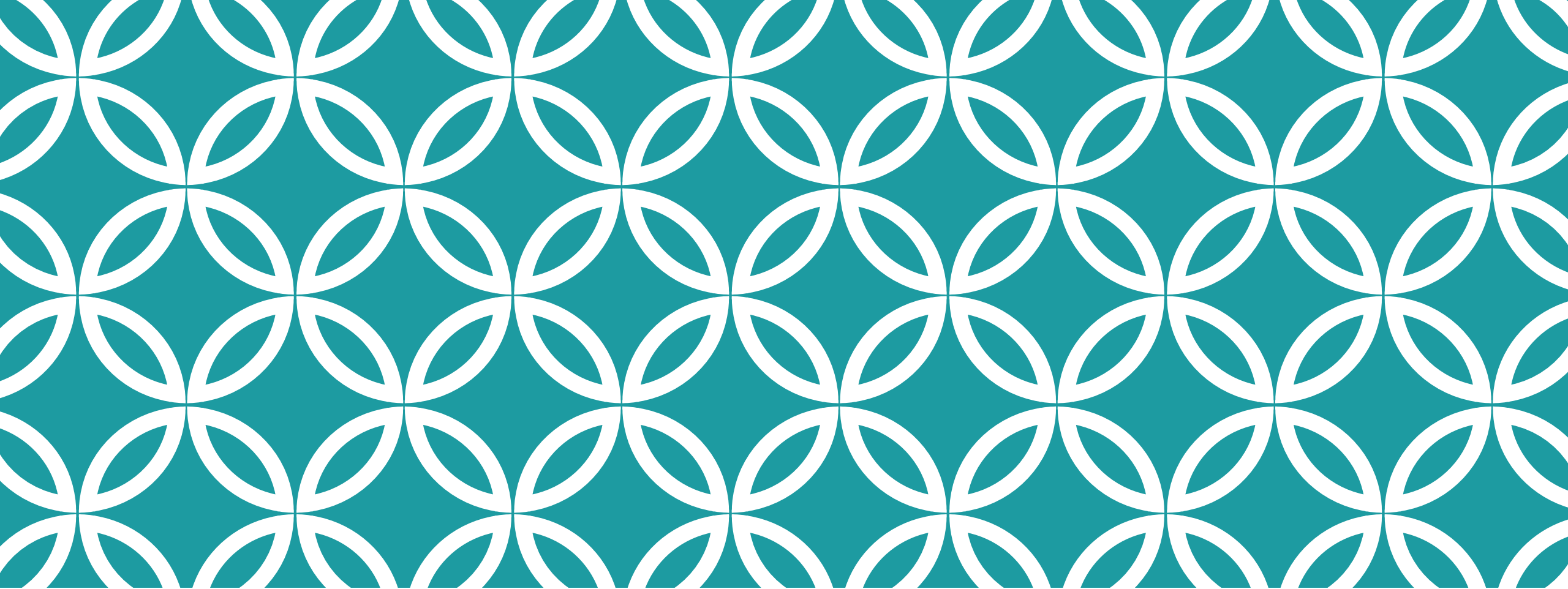
#

② 导入 main.js

```
// ----- main.js -----
import fmt, { withTax, TAX_RATE }
  from './utils.js';

console.log(fmt(withTax(100))); //
¥113.00
console.log(TAX_RATE);          //
0.13

// package.json (Node 中必须)
// { "type": "module" }
```



五、异步编程

目标：模拟延迟加载，了解代码的执行顺序

为什么需要异步

- JS 是单线程：同一时刻只能做一件事，一件事卡住后面全等
- 耗时操作（网络请求、读文件、定时器）如果同步等待，页面/程序会"假死"
- 异步的做法：把耗时操作挂起，先执行后面的代码，完成后再回来通知
- 最原始的异步方案是回调函数；嵌套一多就成了"回调地狱"，难以维护

```
// 回调嵌套 → 回调地狱 (Callback Hell)
getData(function (a) {
  getDetail(a, function (b) {
    getComment(b, function (c) {
      console.log(c);          // 层层缩进，越来越难维护
    });
  });
});

// Promise / async-await 就是为了让这段代码「平」下来
//
```

async 用于定义异步函数

- 一个函数前面加上 `async`，它就变成异步函数。异步函数的特点是：
- 调用后返回一个 `Promise`。
- 函数内部可以使用 `await`。
- `await` 会等待一个 `Promise` 完成，并取出它的结果。
- 如果函数内部抛出错误，返回的 `Promise` 会变成 `rejected`。

Promise 基础

- Promise 是一个“承诺未来会给结果”的对象，用来包装异步操作。Promise 把异步操作包装成一个对象，可以像写同步代码一样链式处理结果。
- 三种状态：pending（进行中）→ fulfilled（成功）/ rejected（失败）。状态不可逆：一旦从 pending 落定，就永远不会再变
- 创建：new Promise((resolve, reject) => { ... }); 成功调 resolve(), 失败调 reject(), 两个函数是 Promise 内部用来“改变 Promise 状态”和“传递结果”的开关。resolve() 表示：事情成功了，这是结果。reject() 表示：事情失败了，这是失败原因。它们是 Promise 构造函数自动传给你的

// 最常用的 Promise: 模拟延迟

```
const delay = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
```

```
const loadBooks = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    const ok = true;  
    ok ? resolve(['三体', '活着']) : reject(new Error('加载失败'));  
  }, 1000);  
});
```

```
console.log(loadBooks); // 该语句是立刻执行的，那一刻 Promise 还是 pending。Promise { <pending> }  
loadBooks.then(console.log); // 1 秒后 → ['三体', '活着']  
//注册回调,.then 的回调(console.log)才会被放进微任务队列;微任务要等所有同步代码跑完才执行
```

1. 把 Promise 状态从 pending 改成 fulfilled

2. 把 ['三体', '活着'] 作为成功结果保存起来

3. 通知所有 .then 的回调，可以执行了

resolve 传进去的值，会变成 .then 回调的参数。

1. 把 Promise 状态从 pending 改成 rejected

2. 把 new Error('加载失败') 作为失败原因保存起来

3. 通知所有 .catch 或 .then 的第二个回调，可以执行了

reject 传进去的值，会变成 .catch 回调的参数。

async / await

- async 函数：返回值会自动包装成 Promise，所以调用后仍可用 .then()
- await：暂停等待右侧 Promise 完成，拿到结果再往下走（只能在 async 函数内使用）
- 好处：写法和同步代码一样，没有嵌套，错误处理也能用 try...catch
- ⚠ 忘了写 await，拿到的是一个 Promise 对象而不是里面的值

```
const delay = (ms) => new Promise((r) => setTimeout(r, ms));

async function main() {
  console.log('开始');
  const books = await loadBooks();      // 停在这里等结果
  console.log('拿到', books);
  await delay(500);
  console.log('结束');
  return books;                        // 自动包成 Promise
}

main().then((books) => console.log('完成'));
// 开始 → (1 秒) → 拿到 → (0.5 秒) → 结束 → 完成
```

执行顺序：宏任务与微任务

- 执行顺序：同步代码 → 清空微任务队列 → 取一个宏任务 → 再清空微任务……
- 微任务：Promise.then / catch / finally （优先级更高）
- 宏任务：setTimeout / setInterval、整体 script （优先级更低）
- 看到 setTimeout(fn, 0)，不要以为它会立刻执行 —— 它排在微任务后面

① 经典顺序题（先自己算一遍）

```
console.log('1');           // 同步
setTimeout(() => console.log('2'), 0); // 宏任务
Promise.resolve()
  .then(() => console.log('3')); // 微任务
console.log('4');           // 同步
```

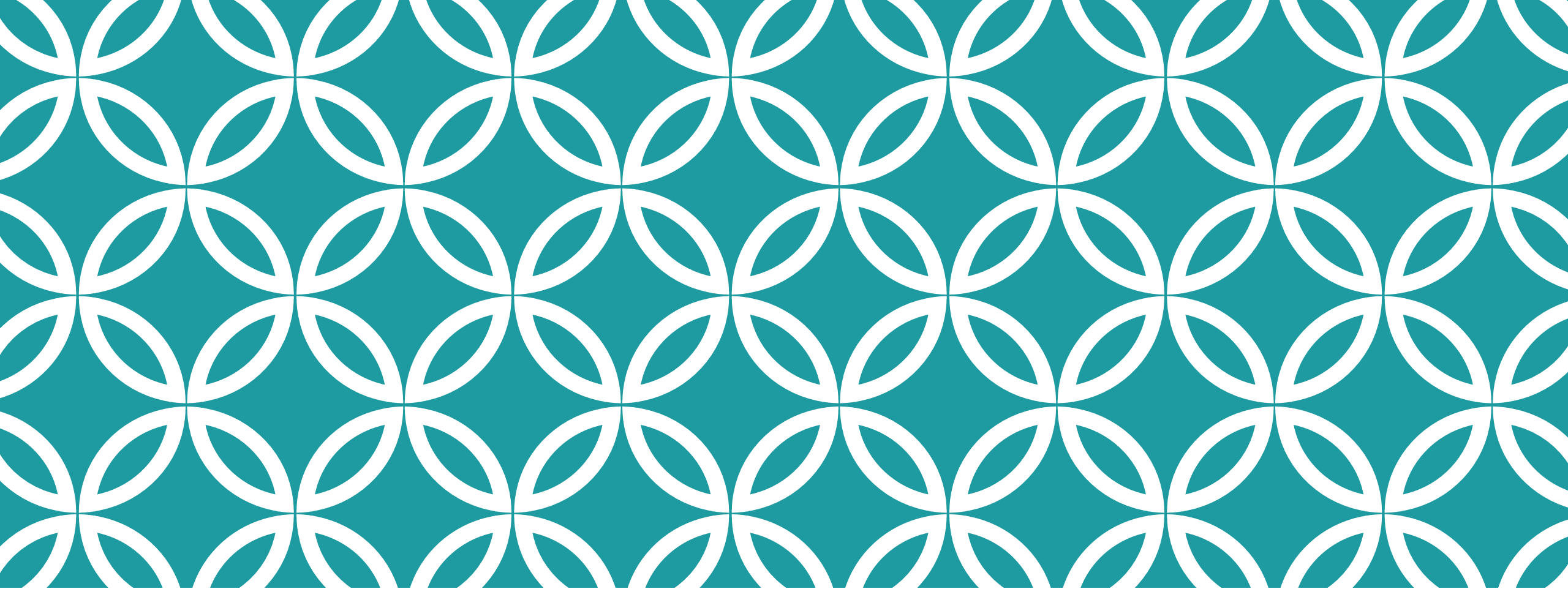
// 正确输出：1 4 3 2
// 错误直觉：1 2 3 4

② 拆解

- ① 同步执行：打印 1、4
- ② 同步结束 → 清空微任务队列：打印 3
- ③ 微任务清空后 → 执行宏任务：打印 2

口诀：

同步 → 微 (Promise) → 宏 (setTimeout)
每一轮宏任务之后，都会把微任务清空一遍



六、调试与错误处理

目标：让程序出错时不崩，并且能定位到出错的那一行

错误类型与 try...catch

- `SyntaxError` 语法错误（代码写错，通常运行前就报）
- `ReferenceError` 引用了不存在的变量 `TypeError` 类型用错（如给 `null` 取属性）
- `try` { 可能出错的代码 } `catch` (err) { 处理 } `finally` { 一定执行 }
- 不捕获的后果：程序直接中断，后面的代码一行都不会执行

```
try {  
  const data = JSON.parse('{ bad json }'); // 抛 SyntaxError  
  console.log(data.title);                 // 出错 → 这行不会执行  
} catch (err) {  
  console.error('解析失败: ', err.message); // 给出可读提示  
} finally {  
  console.log('无论如何都会执行');  
}  
console.log('程序继续');                    // ☒ 没崩，能走到这里
```

then / catch 链式调用

- then 接收上一步的结果，它返回的新值会传给下一个 then
- catch 捕获链上任意环节的错误 —— 错误会"冒泡"到最近的 catch
- finally 无论成功失败都会执行，适合收尾（关闭加载动画）

```
loadBooks
  .then((list) => {
    console.log('拿到', list);
    return list.map((t) => ({ title: t })); // 返回值传给下一个 then
  })
  .then((books) => {
    console.log('转换后', books);
  })
  .catch((err) => { // 上面任一步出错都进这里
    console.error('出错了: ', err.message);
  })
  .finally(() => {
    console.log('无论成败都会执行');
  });
```

#

throw new Error() 主动抛错

- throw 用来主动抛出错误，最常见的场景是参数校验
- new Error(...) 携带可读信息，用 err.message 取出；err.name 默认是 "Error"
- 一旦 throw，当前函数立即停止，错误向上冒泡到最近的 try...catch
- 好处：把"静默失败"变成"明确报错"，出问题能第一时间定位

```
function findById(list, id) {  
  const item = list.find((it) => it.id === id);  
  if (!item) {  
    throw new Error(`找不到 id 为 ${id} 的记录`); // 主动抛错  
  }  
  return item;  
}  
  
try {  
  findById(books, 999);  
} catch (err) {  
  console.error(err.name, err.message);  
  // Error 找不到 id 为 999 的记录  
}
```

异步中的错误处理

- await 外面必须包 try...catch, 否则 Promise 的失败没人接收
- Promise 链用 .catch() 兜底, 写在最后即可覆盖整条链
- 漏写的后果: 控制台出现 UnhandledPromiseRejection, 程序静默失败 —— 最难查
- 建议: 每个 async 主流程最外层都包一层 try...catch

```
// ☒ async / await: 用 try...catch
async function main() {
  try {
    const books = await loadBooks();    // 失败会跳到 catch
    console.log(books);
  } catch (err) {
    console.error('加载失败: ', err.message);
  }
}

// ☒ Promise 链: 用 .catch()
loadBooks().then(console.log).catch(console.error);

// ☒ 没写 catch → UnhandledPromiseRejection, 静默失败
#
```

console 调试输出

- console.log() 普通输出，可一次打印多个值，用逗号分隔
- console.table() 把对象数组渲染成表格
- console.dir() 展开对象的完整结构，看嵌套对象尤其有用
- console.error() / console.warn() 带级别，在控制台里可以筛选

```
const goods = [  
  { name: '笔记本', price: 5000, stock: 3 },  
  { name: '鼠标', price: 80, stock: 0 },  
];  
console.log('共', goods.length, '件');           // 共 2 件  
console.table(goods);                             // 表格形式，一眼看清字段  
console.dir(goods[0], { depth: null });           // 展开结构
```

// console.table 输出效果:

```
//  
//  
//

| (index) | name | price | stock |
|---------|------|-------|-------|
| 0       | 笔记本  | 5000  | 3     |

  
//  
//
```

#

debugger 断点调试

- 在代码里写一行 debugger;，运行到这一行会自动暂停
- Step Over (F10)：执行当前行，不进入被调用的函数内部
- Step Into (F11)：进入被调用的函数内部，逐行查看
- Continue (F8)：继续运行到下一个断点或结束 Step Out (Shift+F11)：跳出当前函数
- 暂停时可以：鼠标悬停看变量实时值、在 Console 里手动取值、在 Watch 添加表达式

```
function total(list) {  
  let sum = 0;  
  for (const g of list) {  
    debugger;           // ← 每次循环都会在这里暂停  
    sum += g.price;      // 悬停可看到 sum 与 g 的实时值  
  }  
  return sum;  
}  
total(goods)
```

// 在HbuilderX中，设置断点（注意双击是设断点，单击是设书签，图标也不同），选择“使用node调试当前文件”，则可以调出调试页面，设置必要的监视对象

Call Stack 与 Scope 面板

- Call Stack（调用堆栈）：显示当前函数的调用链，最上面的是正在执行的函数
- Scope（作用域）：Local 当前函数的局部变量；Closure 闭包捕获的变量；Global 全局变量
- 看不懂报错信息时，先看 Call Stack —— 它能告诉你"是谁调用了谁"
- 作业要求：截图必须同时包含控制台输出、Call Stack 与 Scope 面板

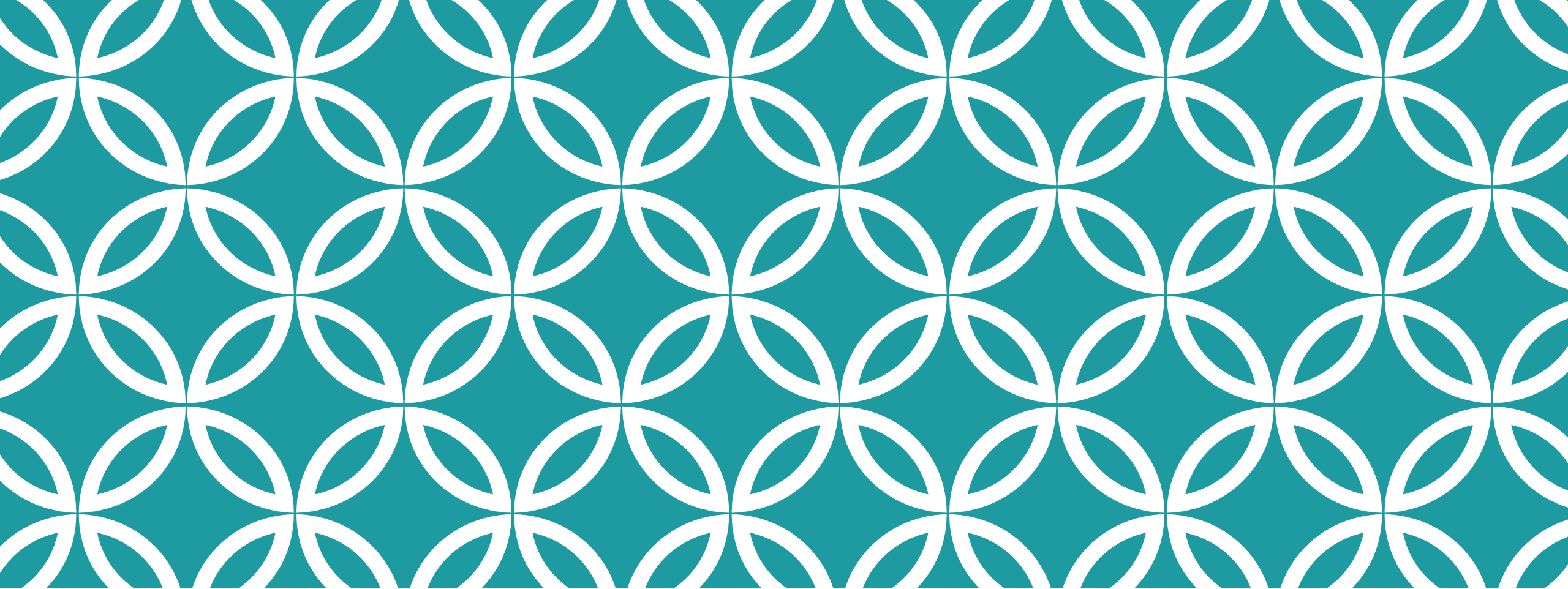
Call Stack

▶ total(list)
main()
(anonymous)

↑ 栈顶 = 正在执行

Scope

Local list: Array(3) sum: 5080 g: {name:"鼠标", price:80}
Closure (无闭包时此区域不显示)
Global goods: Array(3) console: ...



七、阶段实战任务

独立完成一个 JavaScript 数据处理工具（Node.js 或浏览器控制台均可）

任务总览与能力地图

- 任务：独立完成一个 JavaScript 数据处理工具，HbuilderX、Node.js 或浏览器控制台三选一
- 五组要求，逐条对应前面的模块：
 - ① 数据定义 → 模块一（变量与类型）
 - ② 数组与对象操作 → 模块三（map / filter / find / reduce / forEach）
 - ③ ES6 语法应用 → 模块四（模板字符串 / 箭头函数 / 解构 / 扩展）
 - ④ 异步编程 → 模块五（Promise / async / await）
 - ⑤ 错误处理与调试 → 模块六（try...catch / throw / debugger / console.table）



要求（一）数据定义

- ① 定义一个数组，里面不少于 5 个对象
- ② 每个对象至少 4 个字段
- ③ 字段类型需覆盖字符串、数字、布尔，或包含一个嵌套对象
- ④ 必须使用解构赋值获取字段（不能全是 obj.xxx)
- ⑤ 文件顶部用注释写清每个字段的含义

```
/**
 * books: 图书列表
 * id      编号 (数字, 唯一)
 * title   书名 (字符串)
 * price   单价 (数字, 元)
 * stock   库存 (数字, 本)
 * tags    分类标签 (字符串数组)
 * onSale  是否促销 (布尔)
 */
const books = [
  { id: 1, title: '三体', price: 39, stock: 5, tags: ['科幻'], onSale: true },
  // ... 至少 5 条
];

const [{ title, price }] = books;      //# ④ 解构赋值
```

要求（二） 数组与对象操作

- ① map：把数据映射成另一种结构（如对象 → 展示字符串）
- ② filter：按条件筛选子集 ③ find：按条件找单个对象
- ④ reduce：做聚合计算（总价、平均分、分组计数）
- ⑤ forEach：遍历输出或做副作用
- ⑥ 用扩展运算符 ... 完成一次合并或拷贝

```
// ① 映射成展示用的字符串
const lines = available.map((b) => `《${b.title}》¥${b.price}`);
// ② 筛选有货的图书
const available = books.filter((b) => b.stock > 0);
// ③ 按 id 找一本（找不到返回 undefined）
const one = books.find((b) => b.id === 3);
// ④ 计算库存总码洋
const total = books.reduce((s, b) => s + b.price * b.stock, 0);
// ⑤ 逐条打印
lines.forEach((l) => console.log(l));
// ⑥ 合并与拷贝
const all = [...books, ...[{ id: 9, title: '新书' }]];
```

要求（三）ES6 语法应用

- ① 模板字符串不少于 3 处（拼提示语、报表行、多行说明都算）
- ② 箭头函数作为回调不少于 1 处（数组方法的参数基本都会用到）
- ③ 解构赋值不少于 1 处（对象解构 / 数组解构 / 参数解构均可）
- ④ 建议：变量一律用 const / let，全篇不得出现 var；比较一律用 ===

```
// ① 模板字符串 ×3
console.log(`共 ${books.length} 本图书`);
const line = `《${title}》 售价 ¥${price}`;
console.log(`
  统计报表
  总码洋: ¥${total}
`);

// ② 箭头函数作回调
const cheap = books.filter((b) => b.price < 50);

// ③ 解构 + 参数解构
const { title, price } = books[0];
const show = ({ title, price }) => `${title}: ¥${price}`;
```

要求（四） 异步编程

- ① 写一个模拟延迟函数：Promise + setTimeout，延迟 1~2 秒
- ② 用 async / await 调用它，实现"开始加载 → 等待 → 加载完成"的效果
- ③ 至少再写一段 then / catch 链作对照，体会两种写法的区别
- ④ 输出顺序要能讲清楚：哪几行是同步、哪几行在延迟之后

```
// ① 模拟延迟
const delay = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
const fetchBooks = async () => { await delay(1500); return books; };

// ② async / await
async function main() {
  console.log('开始加载...');
  const list = await fetchBooks();      // 等 1.5 秒
  console.log(`加载完成，共 ${list.length} 本`);
}

// ③ then / catch 对照
fetchBooks().then((l) => console.log(l.length)).catch(console.error);
```

要求（五） 错误处理与调试

- ① 用 try...catch 包裹 await 调用，失败时给出可读提示而不是崩溃
- ② 模拟一个错误场景，用 throw new Error(...) 抛出明确信息
- ③ 代码中至少 1 处 debugger 断点，并能说出在那里观察什么
- ④ 用 console.table() 打印列表、console.dir() 查看单个对象结构

```
async function main() {  
  try {  
    console.log('开始加载...');  
    const list = await fetchBooks();  
    debugger; // ③ 在此观察 list 与下方变量  
    console.table(list); // ④ 表格输出  
    console.dir(list[0]);  
    if (!list.length) throw new Error('图书列表为空'); // ② 主动抛错  
  } catch (err) {  
    console.error('出错了: ', err.message); // ① 捕获  
  }  
}
```

#

交付要求与自查表

■ 交付物：① 一个完整可运行的 .js 文件（含必要注释） ② 运行截图

■ 自查 8 步：

- 1. 文件能否一次跑通、无报错？
- 2. 数据是否 ≥ 5 个对象、每个 ≥ 4 个字段？
- 3. map / filter / find / reduce / forEach 是否各用了至少一次？
- 4. 模板字符串 ≥ 3 、箭头函数 ≥ 1 、解构 ≥ 1 ？
- 5. 有没有出现 var？比较是否都用 === ？
- 6. 每个 await 是否都包在 try...catch 里？ 7. debugger 是否真的能停下来？
- 8. console.table / console.dir 的输出是否正常显示？

推荐项目选题（一）选题与核心数据

序号	选题	核心数据结构
1	学生成绩分析系统	{ name, class, scores:{math,english,js}, passed }
2	图书检索与库存管理	{ id, title, author, price, stock, category }
3	购物车结算引擎	{ id, name, price, qty, selected, tags:[] }
4	员工绩效排名系统	{ name, dept, kpi, attendance, bonus }
5	电影评分过滤系统	{ title, year, rating, genres:[], watched }

推荐项目选题（二）核心逻辑要求

序号	选题	核心逻辑要求
1	学生成绩分析	map+reduce 算总分与平均分； filter 筛选及格/优秀； reduce 按班级分组统计
2	图书检索与库存	filter+includes 关键词模糊搜索； 价格区间筛选； some/every 库存预警； reduce 总价
3	购物车结算	filter+reduce 勾选商品小计； 满减与优惠券规则； 库存不足 throw 抛错； 异步结算
4	员工绩效排名	map 加权综合得分； sort 降序排名； reduce 取部门 Top1； switch 绩效分级
5	电影评分过滤	filter 按评分阈值； includes 按类型过滤； 扩展运算符合并已看状态； reduce 平均分